

COMPSCI 5100

ML & AI for Data Science

Dr. Tanaya Guha

www.tanayag.com

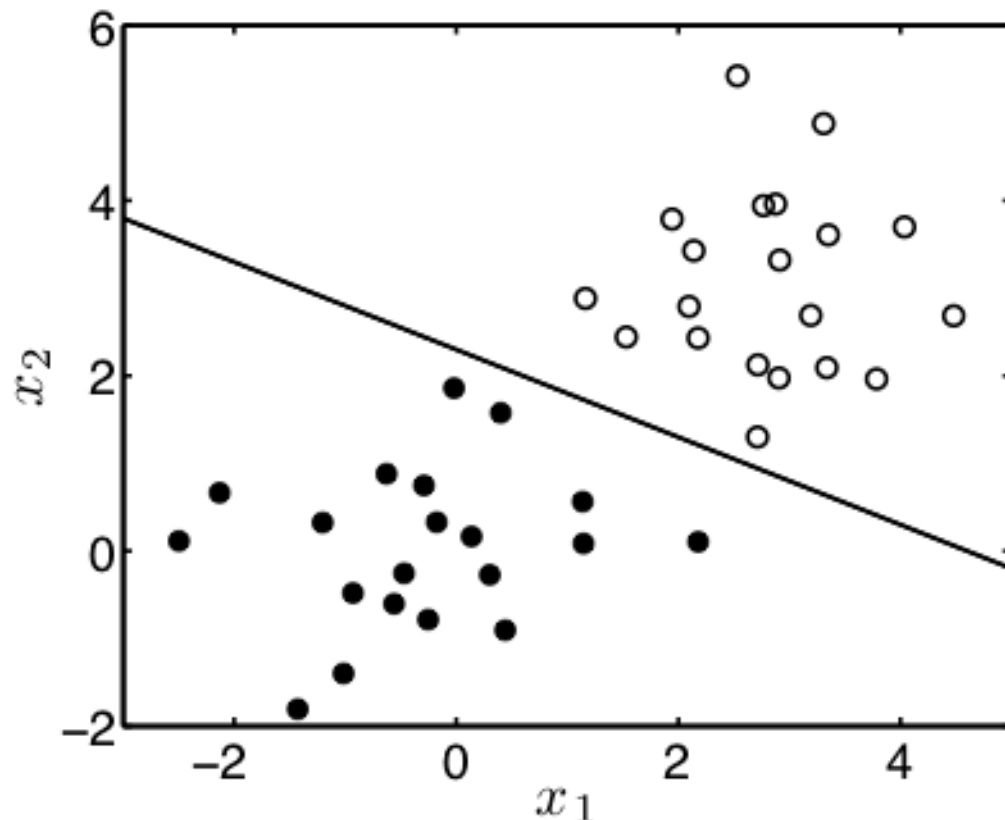
Classification: Part II

Some slides are adapted from those of Dr. Ke Yuan (Glasgow)
and Dr. V. Sancez (Warwick)

Support Vector Machine (SVM)

- Classifiers are learned by minimizing a loss or maximising likelihood
- SVM is no different:
 - Finds a *hyperplane* (decision boundary) in an n-dimensional space separating data points to classes
 - The hyperplane should maximise the distance from data points of either classes
- SVM is a non-probabilistic (binary) classifier

SVM



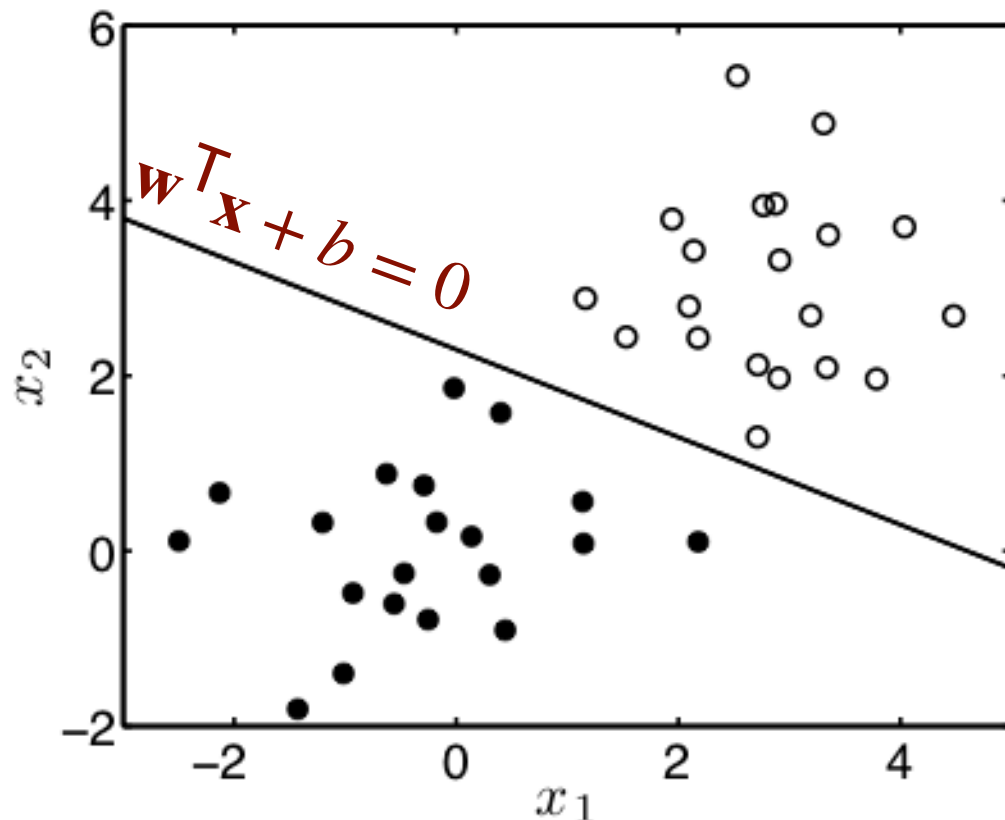
- N data points, each with feature vector: $\mathbf{x} = [x_1, x_2]^T$
- Labels: $y \in \{1, -1\}$

- A linear decision boundary can be represented as a straight line

$$\mathbf{w}^T \mathbf{x} + b = 0$$

- Given training data, our task is to find $\mathbf{w}$ and b

SVM



- N data points, each with feature vector: $\mathbf{x} = [x_1, x_2]^T$
- Labels: $y \in \{1, -1\}$

- Classification:

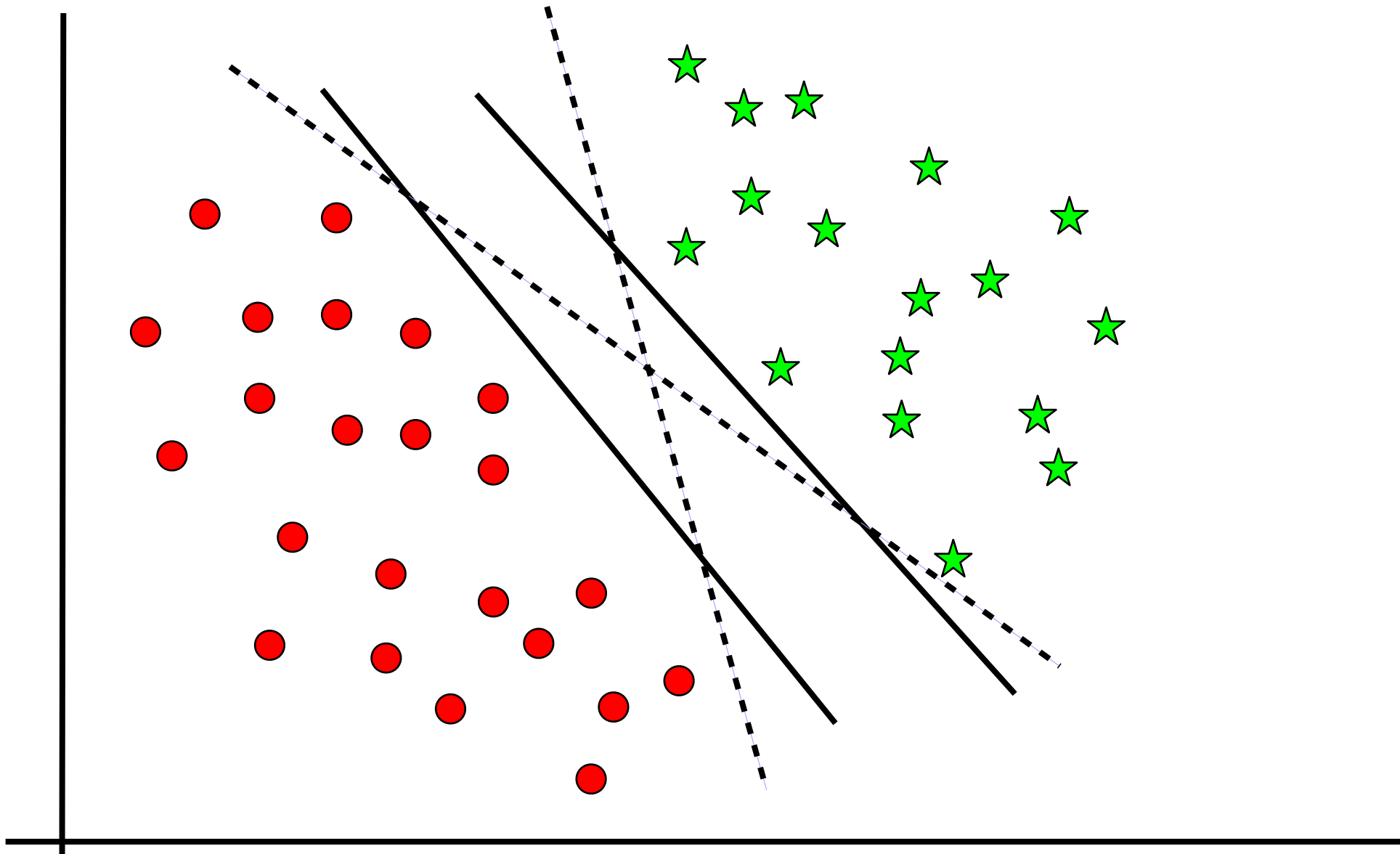
$$\mathbf{w}^T \mathbf{x}_{new} + b > 0 : y_{new} = 1$$

$$\mathbf{w}^T \mathbf{x}_{new} + b < 0 : y_{new} = -1$$

- Classification: $y_{new} = \text{sign}(\mathbf{w}^T \mathbf{x}_{new} + b)$

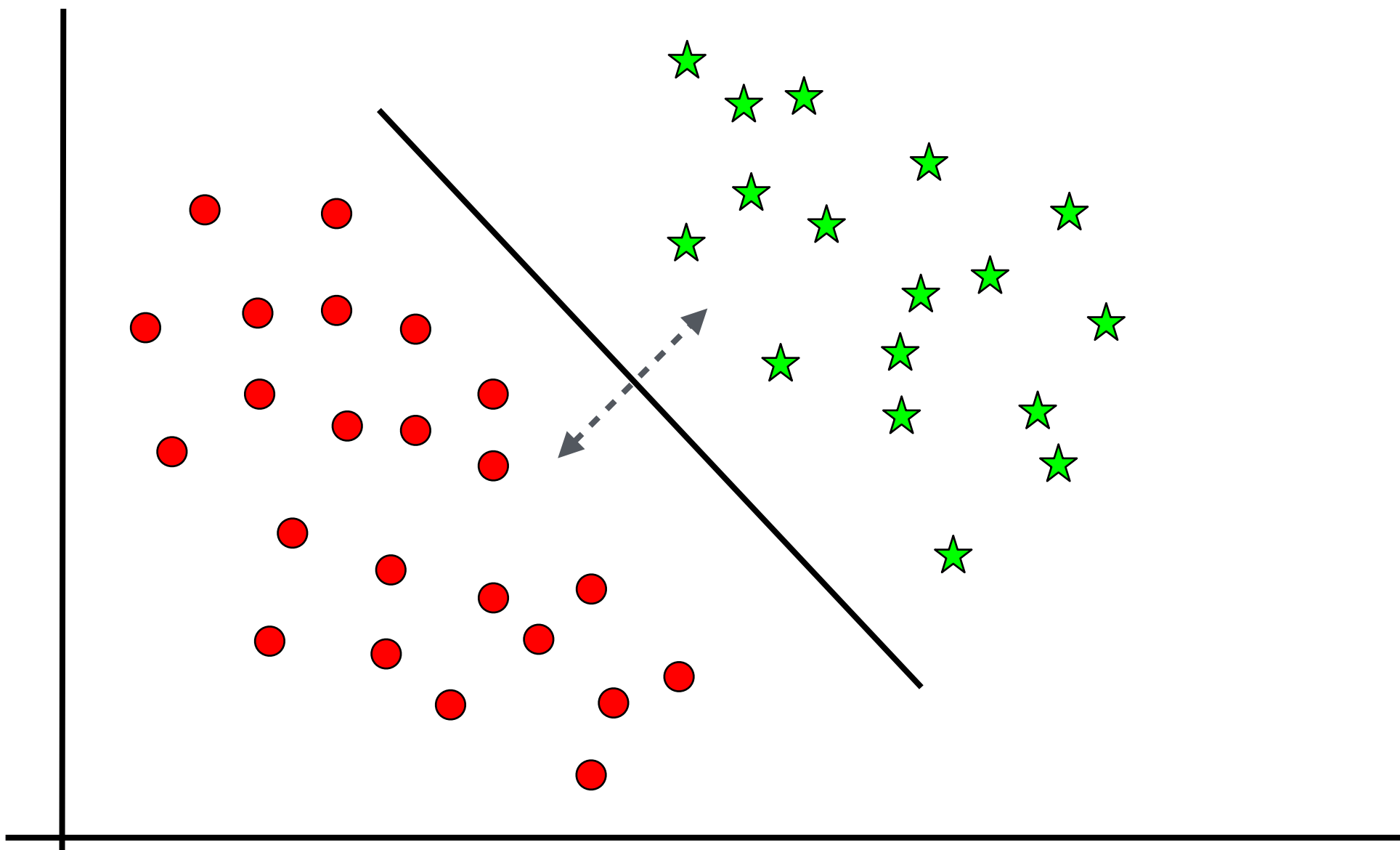
Separating classes

There might be more than one possible classifiers for this dataset. Are they all equally-good?



Separating classes

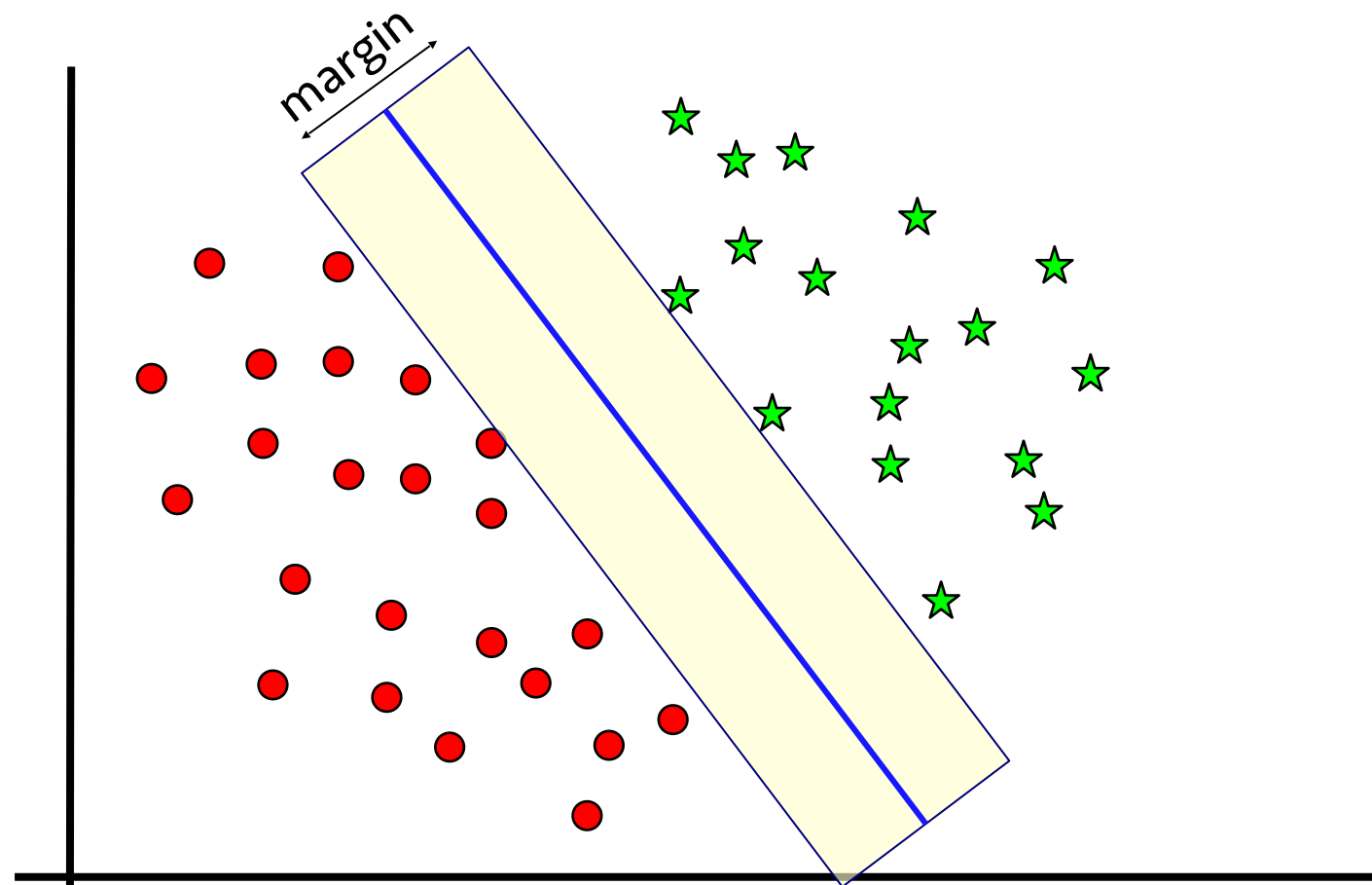
A hyperplane that separates two classes and maximizes the distance to the closest point from either class, i.e., **maximizes the margin** of the classifier.



Separating classes

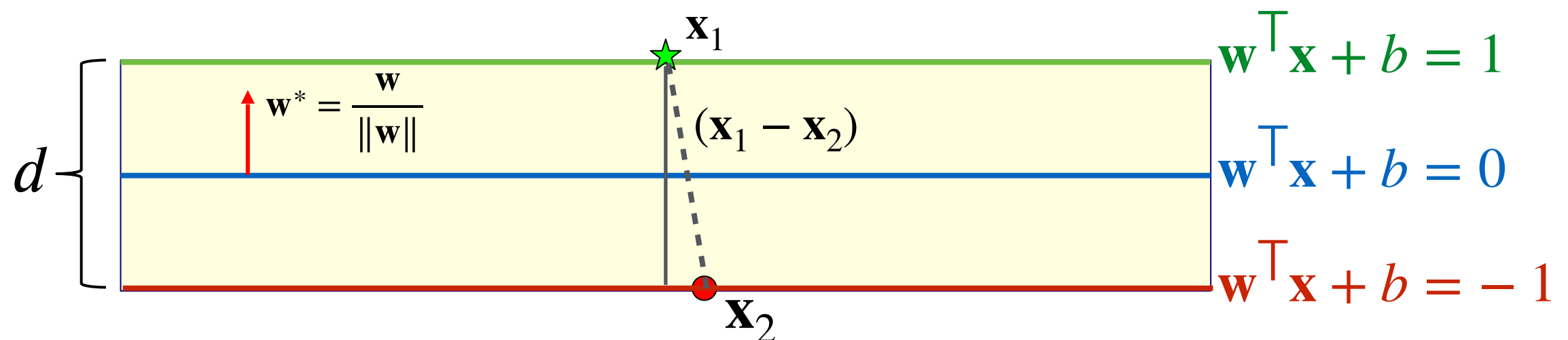
Ensuring that a classifier is *not too close* to any data point leads to better generalization:

- if $\mathbf{x}_{new}$ is close to its training samples, then our classifier performs well
- if $\mathbf{x}_{new}$ is not close to the training data, the margin leaves more room before making an error



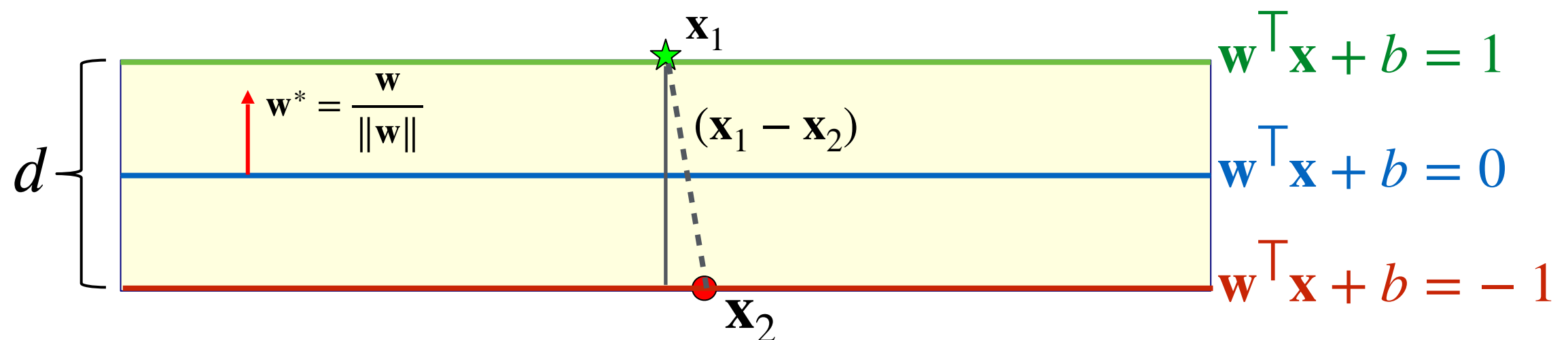
SVM

- $\mathbf{w}$ is orthogonal to the decision hyperplane
- Unit vector in the direction of $\mathbf{w}$: $\hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|}$
- Distance between $\mathbf{x}_1$ to $\mathbf{x}_2$: $(\mathbf{x}_1 - \mathbf{x}_2)$
- d is the projection of $(\mathbf{x}_1 - \mathbf{x}_2)$ in the direction of $\hat{\mathbf{w}}$
- $d = \hat{\mathbf{w}}^T (\mathbf{x}_1 - \mathbf{x}_2) = \frac{1}{\|\mathbf{w}\|} \mathbf{w}^T (\mathbf{x}_1 - \mathbf{x}_2)$



SVM

- maximize d i.e., maximize $\frac{1}{\|\mathbf{w}\|}$
- same as minimising $\|\mathbf{w}\|$
- same as minimising $\frac{1}{2}\|\mathbf{w}\|^2$ with respect to some constraints



SVM

- Minimise $\frac{1}{2} \|\mathbf{w}\|^2$ with respect to some constraints
- **Constraints:**
 - For $\mathbf{x}_i$ with $y_i = 1$: $\mathbf{w}^T \mathbf{x}_i + b \geq 1$
 - For $\mathbf{x}_i$ with $y_i = -1$: $\mathbf{w}^T \mathbf{x}_i + b \leq -1$
 - Combining: $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1$
- **Optimize:**
 $\frac{1}{2} \|\mathbf{w}\|^2$ subject to $y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$

SVM Loss

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 \text{ subject to } y_i \mathbf{w}^T \mathbf{x}_i \geq 1 \quad \forall i$$

same as

$$\min_{\mathbf{w}} \frac{1}{2} \|\mathbf{w}\|^2 - \sum_i \alpha_i (y_i \mathbf{w}^T \mathbf{x}_i - 1) \text{ s.t. } \alpha_i \geq 0$$

More commonly used:

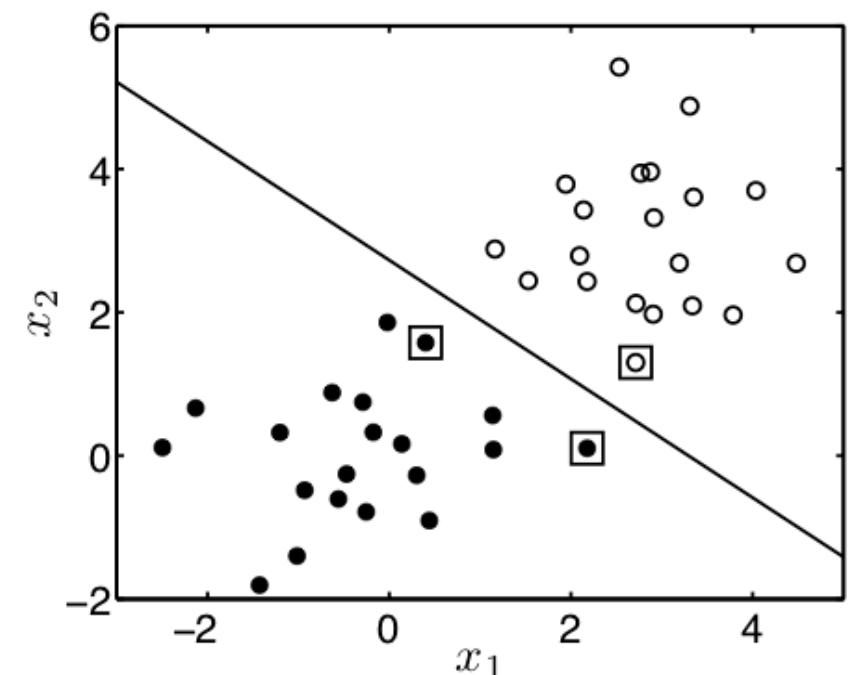
$$\min_{\mathbf{w}} \sum_i \max \left\{ 0, 1 - y_i \mathbf{w}^T \mathbf{x}_i \right\} + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

L2-regularised Hinge loss

Can be solved using standard solvers.

Support vectors

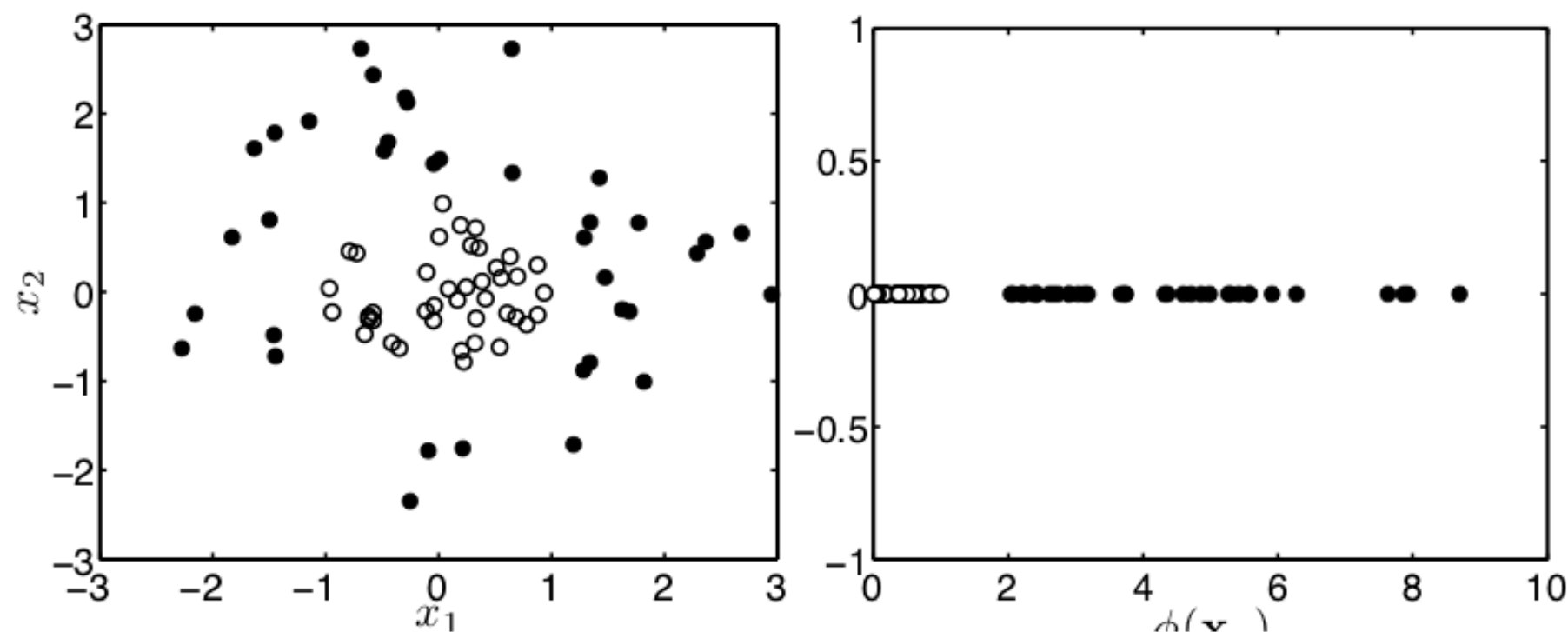
- Prediction of hyperplane only depends on a small number of data points.
 - A small number of non-zero weights
- These are called **support vectors**
- The constraints we used requires all training samples to be correctly classified - this creates a **hard margin**



Kernel trick

- So far we learned a linear classifier. What if the data is not linearly separable?
- Introduce non-linearity
 - Transform data/features to another space through a non-linear kernel
 - Hope: data is linearly separable in the new feature space
 - Example: $k(\mathbf{x}) = x_1^2 + \dots x_d^2$

Kernel trick



- We can transform the data e.g.:

$$\phi(\mathbf{x}_n) = x_{n1}^2 + x_{n2}^2$$

- So that it can be separated with a straight line.
- And use $\phi(\mathbf{x}_n)$ instead of $\mathbf{x}_n$ in our optimisation.

Multiclass SVM

- **One vs. all:** Train one binary classifier for each class, considering the samples of that class as 'positive' and all others as 'negative'
- Let's consider C classes.
 - For each class c , we compute $\mathbf{w}_c^T \mathbf{x}_i$
 - For a datapoint belonging to class 1, we have predictions: $\mathbf{z}_i = \{1, -1, \dots, -1\}$
 - $\mathbf{z}_i = \mathbf{W} \mathbf{x}_i$, where $\mathbf{W}$ has $\mathbf{w}_1, \dots, \mathbf{w}_C$ as rows

Multiclass SVM

- Let's consider C classes.
 - For each class c , we compute $\mathbf{w}_c^T \mathbf{x}_i$
 - For a datapoint belonging to class 1, we have predictions: $\mathbf{z}_i = \{1, -1, \dots, -1\}$
 - $\mathbf{z}_i = \mathbf{W} \mathbf{x}_i$, where $\mathbf{W}$ has $\mathbf{w}_1, \dots, \mathbf{w}_C$ as rows
- Multiclass SVM Loss:

$$\min_{\mathbf{W}} \sum_i \max_{c \neq y_i} \left\{ \max \left\{ 0, 1 - \mathbf{w}_{y_i}^T \mathbf{x}_i + \mathbf{w}_c^T \mathbf{x}_i \right\} \right\} + \frac{\lambda}{2} \|\mathbf{W}\|_F^2$$

Summary

- SVM is a *maximum-margin classifier* for 2 classes: choose the farthest hyperplane from both classes
 - Parameter λ controls trade-off between having a large margin and classifying examples correctly
- Powerful classifier, can model complex data
 - Kernel trick is very useful
 - Can be extended to multi class
 - Was state-of-the-art before deep neural networks